WEEK 11b · PHASE 5

Unity Catalog
Governance & Security

GRANT/REVOKE · Masking · RLS · ABAC · Lineage

- Unity Catalog hierarchy: metastore → catalog → schema → table
- Managed vs external tables
- GRANT, REVOKE, DENY — privilege model
- Column masking — hide sensitive data
- Row-level security (RLS) — filter by user
- Attribute-Based Access Control (ABAC)
- Data lineage & audit logs
- Practice Questions (10 Qs)

Table of Contents

1. Unity Catalog Architecture 3
2. Managed vs External Tables 5
3. Privilege Model — GRANT & REVOKE 7
4. DENY & Privilege Hierarchy 10
5. Column Masking 12
6. Row-Level Security (RLS) 14
7. Attribute-Based Access Control (ABAC) 16
8. Data Lineage & Audit Logs 17
9. Practice Questions 18
10. Answer Key ..... 20 (bonus)

WK 11b **Unity Catalog Architecture** Three-level namespace & object hierarchy

1. Unity Catalog Architecture

Unity Catalog is the unified governance layer for all data assets in Databricks. It provides a three-level namespace (catalog.schema.table), centralized access control, data lineage, and audit logging across all workspaces in an account.

1.1 Object Hierarchy

Level	Object	Description	Example
Account	Metastore	One per region; root of all catalogs	my_company_metastore
Level 1	Catalog	Top namespace; maps to a business domain	prod_catalog, dev_catalog
Level 2	Schema	Namespace within catalog; groups related tables	bronze, silver, gold
Level 3	Table / View	Individual data objects	orders, customers, v_daily_sales
Level 3	Volume	Managed non-tabular file storage	raw_files, documents
Level 3	Function	SQL UDFs, MLflow models	mask_ssn(), predict_churn()
Special	External Location	Governs access to cloud storage paths	s3://bucket/raw/
Special	Storage Credential	Cloud auth for external locations	azure_managed_identity

1.2 Three-Level Namespace

■ **Three-level namespace usage**

```
-- Reference objects with three-part name
SELECT * FROM prod_catalog.silver.orders;

-- Set default catalog for session
USE CATALOG prod_catalog;
SELECT * FROM silver.orders;  -- catalog implied

-- Set default schema
USE CATALOG prod_catalog;
USE SCHEMA silver;
SELECT * FROM orders;  -- catalog + schema implied

-- Show available catalogs
SHOW CATALOGS;
SHOW SCHEMAS IN prod_catalog;
SHOW TABLES IN prod_catalog.silver;
```

1.3 Principals in Unity Catalog

Principal Type	Description	Used For
User	Individual Databricks account user (email)	Personal access, developer grants
Service Principal	Non-human identity (app/automated job)	Production job execution, CI/CD
Group	Collection of users and/or service principals	Bulk permission management
Account Group	Group at account level (spans workspaces)	Cross-workspace access control

2. Managed vs External Tables

Unity Catalog distinguishes between managed tables (Unity Catalog owns both metadata and data) and external tables (Unity Catalog owns only metadata; data lives in external cloud storage).

Feature	Managed Table	External Table
Data location	Unity Catalog managed storage	External cloud path (S3, ADLS, GCS)
DROP TABLE behavior	Deletes BOTH metadata AND data files	Deletes ONLY metadata; data files remain
Creation syntax	CREATE TABLE … USING DELTA	CREATE TABLE … LOCATION 's3://…'
Governance	Full UC governance	Full UC governance (same privileges)
Best for	New Lakehouse tables	Migration from legacy; shared data
External Location required	No (UC manages path)	Yes — external location must exist first

■ **Managed vs external table creation**

```
-- Create MANAGED table (UC manages data location)
CREATE TABLE prod_catalog.silver.orders (
  order_id   STRING NOT NULL,
  customer_id STRING,
  amount     DOUBLE,
  order_date DATE
) USING DELTA;

-- Create EXTERNAL table (data at custom path)
CREATE TABLE prod_catalog.silver.legacy_orders
USING DELTA
LOCATION "abfss://silver@storage.dfs.core.windows.net/orders/";

-- IMPORTANT: DROP TABLE behavior differs!
-- DROP TABLE prod_catalog.silver.orders;
--   Managed: data files permanently deleted
--   External: only Hive metadata removed; files stay
```

■ **Exam Focus** Exam: DROP TABLE on a managed table PERMANENTLY deletes data. DROP TABLE on an external table only removes the metadata — data files are preserved.

3. Privilege Model — GRANT & REVOKE

Unity Catalog uses a privilege-based access model. Privileges are granted on securable objects (catalogs, schemas, tables, views, volumes, functions, external locations) to principals (users, groups, service principals).

3.1 Privilege Types by Object

Object	Key Privileges	Description
Catalog	USE CATALOG	Required to access any object in the catalog
Schema	USE SCHEMA, CREATE TABLE, CREATE VIEW	Required to use schema; create objects
Table / View	SELECT, MODIFY, READ FILES, WRITE FILES	Read/write data access
Table	ALL PRIVILEGES	Grant all privileges at once
External Location	READ FILES, WRITE FILES, CREATE EXTERNAL TABLE	Cloud storage access
Storage Credential	CREATE EXTERNAL LOCATION	Use credential to create locations
Function	EXECUTE	Run a SQL function or model
Volume	READ VOLUME, WRITE VOLUME	Read/write non-tabular files

3.2 GRANT Syntax

■ **GRANT syntax examples**

```
-- Grant SELECT on a table to a group
GRANT SELECT ON TABLE prod_catalog.silver.orders
TO `data_analysts`;

-- Grant multiple privileges at once
GRANT SELECT, MODIFY ON TABLE prod_catalog.silver.orders
TO `data_engineers`;

-- Grant USE on catalog and schema (required for table access)
GRANT USE CATALOG ON CATALOG prod_catalog TO `data_analysts`;
GRANT USE SCHEMA  ON SCHEMA prod_catalog.silver TO `data_analysts`;

-- Grant CREATE TABLE on a schema
GRANT CREATE TABLE ON SCHEMA prod_catalog.bronze TO `data_engineers`;

-- Grant ALL PRIVILEGES on a table
GRANT ALL PRIVILEGES ON TABLE prod_catalog.gold.summary TO `admins`;

-- Grant to a user (by email)
GRANT SELECT ON TABLE prod_catalog.silver.orders
TO `analyst@company.com`;

-- Grant to a service principal
GRANT MODIFY ON TABLE prod_catalog.bronze.events
TO `etl-service-principal`;
```

3.3 REVOKE Syntax

■ **REVOKE and SHOW GRANTS**

```
-- Revoke a specific privilege
REVOKE SELECT ON TABLE prod_catalog.silver.orders
```

```
FROM `data_analysts`;

-- Revoke all privileges on an object
REVOKE ALL PRIVILEGES ON TABLE prod_catalog.silver.orders
FROM `former_employee@company.com`;

-- Show current grants on an object
SHOW GRANTS ON TABLE prod_catalog.silver.orders;

-- Show grants held by a principal
SHOW GRANTS TO `data_analysts`;
```

■ **Exam Focus** Exam: To access a table, a principal needs USE CATALOG on the catalog, USE SCHEMA on the schema, AND SELECT on the table. All three are required.

4. DENY & Privilege Hierarchy

Unity Catalog supports explicit DENY to block access even when a broader grant exists. Privileges cascade down the hierarchy — a grant on a catalog applies to all schemas and tables within.

4.1 DENY — Explicit Block

■ **DENY syntax**

```
-- DENY blocks access even if granted elsewhere
-- Example: group has SELECT on catalog, but one table is sensitive

GRANT SELECT ON CATALOG prod_catalog TO `data_analysts`;
-- This grants SELECT on ALL tables in prod_catalog

-- Block access to the PII table specifically
DENY SELECT ON TABLE prod_catalog.silver.customer_pii
TO `data_analysts`;

-- Result: data_analysts can SELECT all tables EXCEPT customer_pii

-- Remove DENY (restore inherited access)
REVOKE DENY SELECT ON TABLE prod_catalog.silver.customer_pii
FROM `data_analysts`;
```

4.2 Privilege Inheritance Rules

Rule	Detail
Inheritance	Privileges granted on a parent object cascade to children
Specificity	More specific grants/denies override general ones
DENY precedence	DENY always overrides GRANT at the same or parent level
Required chain	To read table: USE CATALOG → USE SCHEMA → SELECT on table
Owner	Object owner has all privileges by default; can grant to others

4.3 Common Privilege Grant Patterns

Use Case	Grant Pattern
Read-only analyst access to silver tables	GRANT USE CATALOG, USE SCHEMA ON catalog/schema; GRANT SELECT ON TABLE
Data engineer — full write to bronze	GRANT USE CATALOG; GRANT USE SCHEMA, CREATE TABLE, MODIFY ON SCHEMA
Block sensitive table from broad group	DENY SELECT ON TABLE sensitive_table TO group
Service principal for prod job	GRANT SELECT, MODIFY on specific tables only (least privilege)
Data steward — full control of schema	GRANT ALL PRIVILEGES ON SCHEMA to steward

5. Column Masking

Column masking applies a SQL function to a column at query time, hiding or transforming sensitive values based on the querying user's identity or group membership. Masking is transparent to users — they see masked values, not errors.

5.1 Creating a Column Mask Function

■ **Column masking full example**

```
-- Step 1: Create a masking function
CREATE OR REPLACE FUNCTION prod_catalog.security.mask_ssn(ssn STRING)
RETURN CASE
  WHEN is_account_group_member("pii_admin")
    THEN ssn                        -- admins see real SSN
  ELSE CONCAT("***-**-", RIGHT(ssn, 4))  -- others see masked
END;

-- Step 2: Apply mask to a column
ALTER TABLE prod_catalog.silver.customers
ALTER COLUMN ssn
SET MASK prod_catalog.security.mask_ssn;

-- Step 3: Query — masking is automatic
SELECT customer_id, name, ssn FROM prod_catalog.silver.customers;
-- pii_admin: 123-45-6789
-- analyst:   ***-**-6789
```

5.2 Masking Function Helpers

Function	Returns	Usage in Masking
is_account_group_member("group")	BOOLEAN	Check if querying user is in a group
current_user()	STRING (email)	Get the querying user's email address
is_member("group")	BOOLEAN	Check workspace-level group membership
session_context("tag")	STRING	Read a session tag value

■ **Additional masking examples**

```
-- Mask credit card — show last 4 digits only
CREATE FUNCTION mask_cc(cc STRING)
RETURN CASE
  WHEN is_account_group_member("finance_team")
    THEN cc
  ELSE CONCAT("****-****-****-", RIGHT(cc, 4))
END;

-- Mask email — replace domain for non-admins
CREATE FUNCTION mask_email(email STRING)
RETURN CASE
  WHEN is_account_group_member("data_admins") THEN email
  ELSE CONCAT(LEFT(email, 1), "***@***.***")
END;

-- Remove mask from column
ALTER TABLE prod_catalog.silver.customers
ALTER COLUMN ssn DROP MASK;
```

■ **Exam Focus** Column masks are applied transparently at query time. Users cannot see raw sensitive data — they always get the masked result unless their group membership grants them the unmasked view.

6. Row-Level Security (RLS)

Row Filters restrict which rows a user can see when querying a table. The filter function is applied automatically at query time, returning only rows the current user is authorized to see.

6.1 Creating a Row Filter

■ **Row filter creation and application**

```
-- Step 1: Create row filter function
-- Users see only their own region's data
CREATE OR REPLACE FUNCTION prod_catalog.security.region_filter(region STRING)
RETURN
  is_account_group_member("global_data_team")  -- admins see all
  OR region = session_context("user_region")   -- others see own region
  OR region IS NULL;                           -- nulls visible to all

-- Step 2: Apply filter to table
ALTER TABLE prod_catalog.silver.orders
SET ROW FILTER prod_catalog.security.region_filter ON (region);

-- Step 3: Verify filter
SELECT * FROM prod_catalog.silver.orders;
-- Users see ONLY rows where region matches their session_context
-- global_data_team members see ALL rows
```

6.2 Row Filter vs Column Mask — Key Differences

Feature	Column Mask	Row Filter
What it hides	Values within a column	Entire rows
User experience	Sees row but with masked value	Row does not appear at all
Function input	The column value being masked	The filter column(s)
Function output	Masked value (same type as column)	BOOLEAN (true = row visible)
Use case	Hide SSN, email, credit card	Hide rows for wrong region, wrong team
Applied with	ALTER TABLE … ALTER COLUMN SET MASK	ALTER TABLE … SET ROW FILTER

■ **Department-based row filter**

```
-- Row filter: user sees only their own department's rows
CREATE FUNCTION dept_filter(dept STRING)
RETURN is_account_group_member("hr_admin")
  OR dept = (SELECT department
             FROM prod_catalog.hr.user_dept_map
             WHERE email = current_user());

-- Apply
ALTER TABLE prod_catalog.hr.employee_data
SET ROW FILTER prod_catalog.security.dept_filter ON (department);

-- Remove row filter
ALTER TABLE prod_catalog.hr.employee_data DROP ROW FILTER;
```

7. Attribute-Based Access Control (ABAC)

ABAC extends standard role-based access by making access decisions based on attributes — properties of the user (group membership, location, clearance level) and the data (sensitivity tag, region, classification). In Databricks, ABAC is implemented through row filters and column masks that use user attributes (group membership, session context) as the access decision criteria.

7.1 ABAC vs RBAC Comparison

Aspect	RBAC (Role-Based)	ABAC (Attribute-Based)
Access decision	Based on role/group membership only	Based on user + data attributes together
Granularity	Coarse — all-or-nothing per role	Fine-grained — row/column level
Databricks implementation	GRANT/REVOKE to groups	Row filters + column masks using is_account_group_member()
Use case	General access control	PII, multi-tenant, regional data isolation
Complexity	Simple	Higher — requires filter/mask functions

7.2 ABAC Pattern — Data Classification Tag

■ ABAC data classification filter

```
-- ABAC example: access based on data sensitivity tag
-- Users with "confidential_access" group see CONFIDENTIAL rows
-- Others see only PUBLIC rows

CREATE FUNCTION sensitivity_filter(classification STRING)
RETURN
  classification = "PUBLIC"
  OR (classification = "CONFIDENTIAL"
      AND is_account_group_member("confidential_access"))
  OR (classification = "RESTRICTED"
      AND is_account_group_member("restricted_access"));

ALTER TABLE prod_catalog.silver.documents
SET ROW FILTER prod_catalog.security.sensitivity_filter
ON (data_classification);
```

■ **Exam Focus** Exam: ABAC in Databricks = row filters + column masks that use is_account_group_member() or current_user() to make attribute-based access decisions. The mechanism is the same as RLS/masking — ABAC is the design pattern.

8. Data Lineage & Audit Logs

8.1 Data Lineage in Unity Catalog

Unity Catalog automatically tracks data lineage — which tables, columns, and notebooks contributed to each downstream table. Lineage is visible in the Catalog Explorer UI and queryable via API.

Lineage Type	Tracked	How to View
Table lineage	Which upstream tables a table was derived from	Catalog Explorer → Table → Lineage tab
Column lineage	Which upstream columns flow into each column	Catalog Explorer → Column details → Lineage
Notebook lineage	Which notebook produced/read a table	Lineage graph shows notebook nodes
Job lineage	Which job/pipeline wrote to a table	Lineage includes job run references

8.2 Audit Logs

All Unity Catalog operations generate audit events. These are available in the Databricks audit log and can be queried via system tables.

■ Audit log queries

```
-- Query audit events via system tables (Unity Catalog)
SELECT
  event_time,
  user_identity.email        AS user,
  action_name,
  request_params.table_full_name AS table_name,
  response.status_code
FROM system.access.audit
WHERE action_name IN ("SELECT","INSERT","UPDATE","DELETE","GRANT","REVOKE")
  AND event_time >= current_date() - INTERVAL 7 DAYS
ORDER BY event_time DESC;

-- Common audit action names:
-- commandSubmit, commandFinish (notebook commands)
-- getTable, createTable, deleteTable (catalog operations)
-- grantPermission, revokePermission (access changes)
```

QUIZ **Practice Questions** Week 11b — Unity Catalog Governance & Security

9. Practice Questions

1. A data analyst needs to query the table prod_catalog.silver.orders. What minimum set of privileges is required?

A) SELECT on the table only

B) USE CATALOG on prod_catalog, USE SCHEMA on silver, SELECT on orders

C) USE CATALOG on prod_catalog and SELECT on orders

D) SELECT on the catalog — it cascades down to all tables

2. A team runs DROP TABLE prod_catalog.silver.customers on a MANAGED table. What happens to the data files?

A) Data files are moved to a recycle bin

B) Only the metadata is deleted — data files remain in cloud storage

C) Both metadata and data files are permanently deleted

D) The table is marked inactive but data is retained for 30 days

3. A column mask function returns the raw SSN for members of "pii_admin" and a masked value for others. An analyst who is NOT in "pii_admin" queries the table. What do they see?

A) An error — they cannot query the column

B) The raw SSN value

C) The masked value (e.g., ***-**-6789)

D) NULL for the SSN column

4. A row filter is applied to the orders table using the region column. A user in "global_data_team" queries the table. The filter returns true for all regions for this group. How many rows does the user see?

A) Only rows matching their own region

B) No rows — row filters always restrict access

C) All rows — the filter evaluates to true for their group

D) Only non-null region rows

5. What is the key difference between DENY and REVOKE in Unity Catalog?

A) They are identical — DENY and REVOKE are synonyms

B) REVOKE removes a grant; DENY explicitly blocks access even if granted by a parent object

C) DENY removes the privilege; REVOKE just suspends it temporarily

D) REVOKE applies to groups; DENY applies to individual users only

6. Which function in a column mask checks whether the querying user belongs to a Unity Catalog group?

A) current_user()

B) is_member("group")

C) is_account_group_member("group")

D) user_in_group("group")

7. A column mask is applied to the "email" column. A user runs: SELECT * FROM customers. What happens?

A) The query fails with a permission error on the email column

B) The email column returns NULL for all users

C) The query succeeds; email column shows masked or real value based on the mask function

D) The email column is excluded from SELECT *

8. DROP TABLE is run on an EXTERNAL table. What is preserved?

A) Nothing — both metadata and data files are deleted

B) The data files at the external location are preserved; only metadata is removed

C) The table is archived, not deleted

D) The table schema is preserved but rows are truncated

9. ABAC (Attribute-Based Access Control) in Databricks is implemented using which mechanisms?

A) Special ABAC GRANT statements with attribute conditions

B) Row filters and column masks that evaluate user attributes (group membership, session context)

C) A separate ABAC policy engine outside of Unity Catalog

D) Databricks Secret Scopes with attribute-based keys

10. Unity Catalog automatically tracks which of the following without any manual configuration?

A) Data quality metrics and anomaly detection

B) Table and column lineage — which tables/columns produced downstream tables

C) Query performance metrics and slow query logs

D) Data volume trends over time

ANSWERS **Answer Key** Week 11b Practice Questions

10. Answer Key

#	Ans	Explanation
1	(B)	To access a table in Unity Catalog, a principal needs three privileges: USE CATALOG on the catalog, USE SCHEMA on the schema, and SELECT on the table. Missing any one blocks access.
2	(C)	DROP TABLE on a managed table permanently deletes BOTH the metadata (in Unity Catalog) AND the underlying data files. This is irreversible without time travel within the retention window.
3	(C)	Column masks are transparent — the query succeeds but returns the masked value. The analyst sees ***-**-6789 (or whatever the mask function returns for non-admins). No errors occur.
4	(C)	Row filters return true/false per row. If the filter function returns true for all rows (e.g., because the user is in global_data_team), the user sees all rows. Admins typically bypass row filters this way.
5	(B)	REVOKE removes a previously granted privilege. DENY explicitly blocks access — even if a parent object (e.g., catalog) grants access, a DENY on a child object overrides it for that specific principal.
6	(C)	is_account_group_member("group") checks account-level Unity Catalog group membership. is_member() checks workspace-level groups. For Unity Catalog column masks and row filters, is_account_group_member() is the standard.
7	(C)	Column masks are transparent — SELECT * works normally. The email column is included but its value passes through the mask function. The result depends on the querying user's group membership.
8	(B)	DROP TABLE on an external table removes only the Unity Catalog metadata entry. The actual data files at the LOCATION path are NOT deleted — they remain in cloud storage.
9	(B)	Databricks ABAC uses the same row filter and column mask mechanisms, but the filter/mask functions use is_account_group_member(), current_user(), or session_context() to make access decisions based on user and data attributes.
10	(B)	Unity Catalog automatically captures table and column lineage without any configuration. It tracks which upstream tables, columns, notebooks, and jobs contributed to each downstream object.

11. Week 11b Cheat Sheet

Topic	Key Fact
UC three-level namespace	catalog.schema.table — USE CATALOG + USE SCHEMA + privilege on object
Principals	User / Service Principal / Group / Account Group
Managed table DROP	Deletes BOTH metadata and data files — PERMANENT
External table DROP	Deletes metadata ONLY — data files remain
External table creation	Requires LOCATION clause and External Location grant
GRANT syntax	GRANT ON TO ``
REVOKE	Removes a granted privilege
DENY	Explicitly blocks access; overrides parent grants
Access chain	USE CATALOG + USE SCHEMA + SELECT = minimum to read a table
Column mask	ALTER TABLE … ALTER COLUMN SET MASK ; transparent to users
Row filter	ALTER TABLE … SET ROW FILTER ON (col); TRUE = row visible
is_account_group_member()	Check UC group membership in mask/filter functions
current_user()	Returns querying user's email in mask/filter functions
ABAC in Databricks	Row filters + column masks using user attributes
Data lineage	Auto-tracked by UC — table, column, notebook, job lineage
Audit logs	system.access.audit table — all UC operations logged

BONUS **Advanced UC Patterns** Deep Dive — Volumes, Shares, Exam Prep

A. Unity Catalog Volumes

Volumes are Unity Catalog objects for managing non-tabular files (PDFs, images, JSON files, CSVs not yet loaded). They provide governed access to files without creating Delta tables.

Volume Type	Storage	Use Case
Managed Volume	UC-managed cloud storage	Raw files, checkpoints, model artifacts
External Volume	External cloud path	Existing data lake files, partner data drops

■ Volumes — creation, grants, COPY INTO

```
-- Create managed volume
CREATE VOLUME prod_catalog.bronze.raw_files;

-- Create external volume
CREATE EXTERNAL VOLUME prod_catalog.landing.partner_drop
LOCATION "abfss://landing@storage.dfs.core.windows.net/partner/";

-- Grant access to volume
GRANT READ VOLUME ON VOLUME prod_catalog.bronze.raw_files
TO `data_engineers`;

-- WRITE VOLUME for uploading files
GRANT WRITE VOLUME ON VOLUME prod_catalog.bronze.raw_files
TO `etl_service_principal`;

-- Reference volume path in COPY INTO
COPY INTO prod_catalog.bronze.orders
FROM "/Volumes/prod_catalog/bronze/raw_files/orders/"
FILEFORMAT = JSON;
```

B. Delta Sharing

Delta Sharing is an open protocol for sharing live Delta tables with external organizations without copying data. Unity Catalog manages shares natively.

Concept	Description
Share	A named collection of tables/schemas/volumes to share externally
Recipient	An external organization or user who receives the share
Provider	The organization sharing data (owns the data)
Open Sharing	Recipient uses any Delta Sharing-compatible client
Databricks-to-Databricks	Recipient imports share directly into their UC metastore

■ Delta Sharing setup

```
-- Create a share
CREATE SHARE partner_share
  COMMENT "Shared orders data for Partner Co";

-- Add table to share
ALTER SHARE partner_share
ADD TABLE prod_catalog.gold.daily_revenue;

-- Create recipient
CREATE RECIPIENT partner_co
  COMMENT "Partner Company external recipient";

-- Grant share to recipient
GRANT SELECT ON SHARE partner_share TO RECIPIENT partner_co;

-- Show shares
SHOW SHARES;
SHOW ALL IN SHARE partner_share;
```

C. Complete Privilege Reference

Privilege	Object	Description
USE CATALOG	Catalog	Required to access any object in catalog
USE SCHEMA	Schema	Required to access any object in schema
CREATE CATALOG	Metastore	Create new catalogs
CREATE SCHEMA	Catalog	Create new schemas in catalog
CREATE TABLE	Schema	Create tables in schema
CREATE VIEW	Schema	Create views in schema
CREATE FUNCTION	Schema	Create functions in schema
CREATE VOLUME	Schema	Create volumes in schema
SELECT	Table / View	Read data from table or view
MODIFY	Table	INSERT, UPDATE, DELETE on table
ALL PRIVILEGES	Any object	All privileges on the object
READ FILES	External Location / Volume	Read files at storage path
WRITE FILES	External Location / Volume	Write files at storage path
CREATE EXTERNAL TABLE	External Location	Create external table at location
CREATE EXTERNAL LOCATION	Storage Credential	Create external locations using credential
READ VOLUME	Volume	Read non-tabular files in volume
WRITE VOLUME	Volume	Write non-tabular files to volume
EXECUTE	Function	Run a SQL function or model
APPLY TAG	Any object	Apply metadata tags to objects

D. Domain 7 Exam Keywords Quick Map

Keyword in Question	Answer Points To
"three-level namespace"	catalog.schema.table — USE CATALOG + USE SCHEMA + privilege
"DROP TABLE deletes data"	Managed table — both metadata AND data files deleted
"DROP TABLE keeps data"	External table — only metadata removed; files remain
"hide SSN from non-admins, query still works"	Column masking — ALTER TABLE … ALTER COLUMN SET MASK
"users see only their region's rows"	Row filter — ALTER TABLE … SET ROW FILTER
"block one table despite catalog grant"	DENY SELECT ON TABLE to principal
"access based on group + data attribute"	ABAC pattern — row filter or column mask with is_account_group_member()
"USE CATALOG needed for table access"	Yes — mandatory in UC privilege chain
"auto-track table lineage"	Unity Catalog automatic lineage — no config needed
"query all UC operations"	system.access.audit table
"share data with external org"	Delta Sharing — CREATE SHARE + RECIPIENT
"non-tabular file storage in UC"	Volumes — managed or external
"READ VOLUME privilege"	Required to read files from a UC Volume
"is_account_group_member()"	UC group check in mask/filter functions
"current_user()"	Returns querying user email in mask/filter
"service principal in UC"	Use service principal as a principal in GRANT statements

E. Self-Assessment & Final Key Facts

Week 11b Self-Assessment

- ■ I know the UC hierarchy: metastore → catalog → schema → table/view/volume/function
- ■ I know managed vs external table DROP behavior (managed=delete data; external=keep data)
- ■ I can write GRANT SELECT, USE CATALOG, USE SCHEMA, CREATE TABLE statements
- ■ I understand the 3-privilege chain required for table access
- ■ I know DENY overrides parent GRANTs for a specific principal
- ■ I can create a column mask with is_account_group_member() check
- ■ I can create a row filter that returns BOOLEAN for row visibility
- ■ I understand ABAC = row filters + column masks using user attributes
- ■ I know UC auto-tracks table and column lineage without config
- ■ I know system.access.audit for querying UC audit events

■ **Exam Focus** Domain 7 (Governance & Security) = 15% of exam (~7 questions). Focus on privilege chain, managed vs external table DROP behavior, column masking, and row filters — highest-yield topics.

Phase 5 Complete — Domain 6 + 7 Coverage

Domain	Weight	Week	Key Topics
Domain 6: Troubleshooting & Optimization	10%	11a	Spark UI, skew, OOM, spill, Liquid Clustering, Predictive Opt
Domain 7: Governance & Security	15%	11b	UC hierarchy, GRANT/REVOKE/DENY, masking, RLS, ABAC, lineage